



۸۱۰۱۰۱۴۹۲



شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی

نوت ۹ دی

فصل ششم: *link layer* اسلاید ۶۵ تا ۸۲ [۱]

self learning

دروائع سویچ ها به صورت خودآموز جدول سویچ را پر میکنند. به این شکل عمل میکنند:

اگر یک فریم از یک *interface* آمد، آن مک و آن *interface* را در صورت نبودن در جدول، به جدول اضافه میکند. حال آدرس مک مقصد آن فریم را بررسی میکند، اگر مدخل مربوط به آن در جدول وجود داشت، دو حالت پیش میاید. حالت اول اینکه آدرس مک مقصد روی همان *interface* و *segment* قرار دارد که بسته از آنجا آمده است. (منظور از سگمنت، همان باس یا هاب مشترک منتهی به *interface* است) در این صورت، هر دوی مبدا و مقصد متعلق به یک LAN و سگمنت هستند و قبلا فریم توسط مقصد گرفته شده، پس فریم را دراپ میکند.

حالت دوم اینکه یک *interface* دیگر به غیر ورودی باشد، در این صورت به صورت *selective* به آن فوروارد میکند. اگر مدخل مربوط به آن در جدول نباشد، به همه *interface* ها به غیر از ورودی، میفرستد و به نوعی *flood* میکند. اما تنها اون مقصد مورد نظر فریم را گرفته و مطالعه کرده و در صورت لزوم پاسخ میدهد. بقیه دراپ میکنند. وقتی مقصد فریم پاسخ را فرستاد، مدخل مربوطه به آن درسویچ اضافه میشود.

پس مدخل باشد به صورت *selective* و اگر نباشد به صورت *flood*

این سویچ ها میتوانند به صورت چند لایه و سلسله مراتبی باشند، یعنی یک درخت تشکیل دهند، یعنی چند سویچ خودشان به سویچ بزرگتر با پهنای باند بیشتر وصل باشند.

برای درون سازمان ها، سرور ها و هاست های سازمان به سویچ ها وصل هستند و برای اتصال به شبکه خارجی، روتر ها لازم هستند.

switches vs. routers

هر دو به صورت *store and forward* هستند و بسته ها را گرفته، مطالعه کرده و فوروارد میکنند. سویچ ها فریم ها را مطالعه میکنند و روتر ها دیتاگرام ها را.

هر دو به نوعی *forwarding table* دارند. سوییچ‌ها به صورت *self learning* آن آدرس‌های مک را پر میکنند، و در روتر با کمک الگوریتم‌های روتینگ و به صورت *programmable* اون آدرس‌های *IP* را در جدول پر میکنند. روتر تا لایه شبکه و در سوییچ تا لایه دیتالینک را مطالعه میکند (تا این لایه‌ها در آن‌ها پیاده‌سازی شده) تنها در مبدا و مقصد است که کل ۵ لایه استک پروتکل، مطالعه میشود.

VLAN

انگیزه‌ها:

- برای اینکه کاربری بخواهد در یک ساختار سلسله‌مراتبی که متلا چند سوییچ داریم که خودشان به یک سوییچ مرکزی وصل هستند، بخواهیم آن کاربر را از یک دپارتمان، به یک دپارتمان دیگر جابجا کنیم بدون اینکه سیم کشی شبکه را تغییر دهیم. (و انگار میخواهد هنوز به دپارتمان قبلی متصل باشد؟)
 - اینکه انگار نگذاریم پیام برودکست که در بعضی پروتکل‌ها مثل *ARP* و *DHCP* هست، به همه منتقل و برودکست شود چون ترافیک را خیلی زیاد میکند.
 - بحث امنیت *security/privacy*
 - بحث کارایی *efficiency* شبکه
- سوییچ‌هایی هستند که از قابلیت *vlan* پشتیبانی میکنند و میتوانیم روی یک *lan* چندین تا *vlan* داشته باشیم.

port – based VALN

بر اساس پورت بیاییم تقسیم بندی کنیم. مثلاً بگیم پورت‌های ۱ تا ۸ در یک *vlan* و پورت‌های ۹ تا ۱۶ در یک *vlan* دیگر هستند. اینجامیتوانیم *traffic isolation* داشته باشیم، یعنی ترافیک هر *vlan* فقط در خودش است و به *vlan* کناری نمیرسد.

dynamic membership داریم و میتوانیم به صورت پویا، پورت‌ها را به هر *vlan* مورد نظر منصوب کنیم و تغییر دهیم.

برای اتصال *vlan*‌ها به هم و انتقال فریم از *vlan* به *vlan* دیگر، از روتر استفاده میکند.

vlan‌ها میتوانند روی چند تا سوییچ گسترده شوند!!

حالا برای اتصال بین این *vlan*‌ها که در چند تا سوییچ گسترده شدند، در هر کدام از سوییچ‌های آنها، یک *trunk port* قرار میدهیم و از طریق آن *trunk port* فریم‌های آن *vlan* را بین سوییچ‌هایی که روی آنها گسترده شده، منتقل میکنیم. نکته اینکه اینجا دیگه فریم عادی 802.1 نیست و باید خود *vlan ID* را هم منتقل کنیم با فریم. که به آن

802.1 q میگویند.

مثلا دو بایت برای فیلد *tag protocol identifier* اضافه میکنند برای اینکه بگن از 802.1 q دارن استفاده میکنند که با 00 – 81 پر میشود.

یک فیلد *tag control information* هم هست که با ۱۲ بیتش، اون *vlan ID* مشخص میشود و با ۳ بیت هم انگار اون *priority* مشخص میشود.

MPLS = multiprotocol label switching

اینجا در واقع فکر کنم از شبکه های *virtual circuit* الهام گرفته و به نوعی میخواهد ایده *circuit switch* را در شبکه *packet switch* پیاده سازی کند.

اینجا به جای *prefix match* برای آدرس *IP* که زمانبر است، از *exact match* استفاده میکند که این آن از *VC* آمده و عمل *lookup* در آن بسیار بالاتر است. برای همین یک سرایند *MPLS* به هدر فریم اضافه میکنیم. هدر *MPLS* ۴ بایت است. ۲۰ بیت اول برای *label* است. ۳ بیت بعدی کلاس ترافیک را مشخص میکند که به *Exp* معروف است. یک بیت *S = stack* است برای نشان دادن انتهای یک استک. (گفت برای مواقعی که بیشتر از یک برچسب به یک بسته منصوب میشود ولی یعنی چی؟؟)

یک فیلد ۸ بیتی برای *TTL* هم دارد.

روتر های دارای قابلیت *mpls* مکه به روتر های *LSR = label – switched router* هستند، فقط طبق همون *label* فوروارد میکنند و دیگر به آدرس *IP* کار ندارند.

در واقع *MPLS forwarding table* از *IP forwarding table* متفاوت است.

توی *MPLS* میتونیم بین مبدا و مقصد های مختلف تمایز قائل بشیم و *traffic engineering* داشته باشیم. میتونیم مسیر های بکاپ رو از قبل محاسبه کنیم و در صورت *fail* کردن یک مسیر سریعا از آنها استفاده کنیم. که به آن *fast re – route* میگویند.

در واقع در روتینگ *IP* فقط براساس مقصد عمل میکند.

اما در روتر های *MPLS* میتواند از چند تا مسیر بر اساس آدرس *IP* مبدا و مقصد به یک مقصد بفرستد. در واقع یه جور همان *generalized forwarding* است که البته *MPLS* ۱۰ سال زودتر آمده است.

جدول *MPLS* ۴ بخش دارد.

۱. *in – label* که شماره لیبل فریم در هنگام ورود است.

۲. *out – label* که شماره لیبل فریم هنگام خروج است.

۳. *dest* که همان میزبان مقصد است.

۴. *out interface*

به روتر اول در شبکه *MPLS*، *LER = label edge router* است و در آن *in-label* خالی ست و انگار ۳ تا فیلد دارد. (البته روتر آخر هم گویا *out-label* نداشت ولی استاد چیزی راجع بهش نگفت)

MPLS signaling

در واقع در شبکه‌های ممخابراتی، روترها به کمک سیگنالینگ اطلاعاتی را راجع به وضعیت شبکه بدست میاورند. مثلا پهنای باند و وضعیت لیتک‌ها. درواقع *LER* از تغییر یافته پروتکل‌های *flooding* که *link-state* هستند مثل *OSPF* استفاده میکند تا *bandwidth* لینک‌ها را بفهمد و بفهمد که جثدر رزرو شده ازشون با کمک سیگنالینگ *RSVP-TE* این اطلاعات را بدست میاورد و بعد به کمک همان، جداول فورواردینگ *MPLS* را در هر کدام نصب میکند. روترهای پایین دست هم به کمک پروتکل *LDP = label = distribution protocol* یک برچسب برای شبکه‌های متصل به خود انتخاب میکنند و به روترهای بالادست میدهند.

مراجع

[1] “ch6-slide-65-to-82,” <https://drive.google.com/drive/folders/>, 2022.